

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
KALANKI, KATHMANDU**



**A PROJECT REPORT ON
"WEEKLY SCHEDULE PLANNER: A CONSOLE-BASED SCHEDULE
MANAGEMENT SYSTEM USING FILE HANDLING IN C++"**

Submitted By:

Biraj Lamsal (ACE082BCT028)

Project Supervisor:

Mukunda Raj Joshi

2083/04/18

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my project supervisor, Mukunda Raj Joshi, for his continuous guidance, valuable suggestions, and encouragement throughout the course of this project. I am also thankful to the Department of Electronics and Computer Engineering, Advanced College of Engineering and Management, for providing the opportunity and resources necessary to complete this Object-Oriented Programming project. Finally, I extend my thanks to my friends and family for their constant support and motivation during the development of this schedule planner application.

ABSTRACT

This project presents the design and implementation of a console-based weekly schedule planner developed in C++ to demonstrate core Object-Oriented Programming concepts alongside file handling. The system allows a user to enter personal details, plan activities for each day of the week as time-bound slots, and persist all data to a text file so that Create, Read, Update, and Delete (CRUD) operations can be performed on the stored schedule. A base class, *Me*, stores personal details, while a derived class, *Plan*, inherits from it and manages a per-day collection of schedule slots using arrays of vectors and a custom *Slot* structure. Every change made to the schedule, whether adding, editing, or deleting a slot, is immediately written back to a file named *plan.txt* using file stream operations, ensuring that the stored record always reflects the current state of the schedule. Beyond the core CRUD operations, the program also provides supporting features such as keyword-based activity search, a smart reminder that reports the current and next activity relative to a given time, weekly statistics that aggregate time spent per activity into a simple text-based bar chart, and a short free-text note feature. The project was tested for correct input validation, file writing and formatting, and correct behaviour of each menu option, and the result is a functional, menu-driven schedule management tool that fulfils its objective of applying inheritance, encapsulation, and file handling in a practical, everyday-use console application, with scope for future enhancement such as loading previously saved data on startup and adding a graphical interface.

Keywords: *CRUD Operations, File Handling, Inheritance, Object-Oriented Programming, Schedule Management*

TABLE OF CONTENTS

LIST OF FIGURES

Fig 3.1: Class Design (Me and Plan)

Fig 3.2: Program / Menu Flow

Fig 4.1: Personal Details Entry Output

Fig 4.2: Adding and Viewing a Slot (Create + Read)

Fig 4.3: Weekly Statistics Output

Fig 4.4: Saved plan.txt File Content

LIST OF TABLES

Table 3.1: Class Responsibilities

Table 4.1: Testing Summary

LIST OF ABBREVIATIONS

CRUD : Create, Read, Update, Delete

HH:MM : Hour : Minute (24-hour time format)

IEEE : Institute of Electrical and Electronics Engineers

OOP : Object-Oriented Programming

CHAPTER 1: INTRODUCTION

This chapter discusses the background of the project, the motivation behind it, the problem it addresses, its objectives, and its significance.

1.1 Background

Keeping track of daily and weekly activities is a common personal-organization need, and a simple, file-backed console tool is often sufficient for this purpose without requiring a full application or database. This project was undertaken to design and implement a console-based weekly schedule planner in C++ that demonstrates the practical application of Object-Oriented Programming (OOP) concepts, particularly inheritance and encapsulation, together with file handling for persistent storage. The project follows a two-class design, where a base class stores personal details and a derived class manages a weekly schedule built from day-wise collections of time slots.

1.2 Motivation

The motivation for this project stemmed from the desire to apply Object-Oriented Programming concepts and file handling to a small but genuinely useful, everyday tool rather than a purely abstract exercise. Implementing Create, Read, Update, and Delete operations backed by a real text file offered practical experience with file streams, input validation, and organizing related data using classes, structures, vectors, and maps.

1.3 Problem Statement

Many introductory file-handling exercises only demonstrate a single record being written to or read from a file, and do not address the complexities of maintaining multiple related records, validating user-supplied data such as times, or reflecting updates and deletions back to storage consistently. This project addresses that gap by implementing a schedule planner that stores a full week of time-bound activities, validates time input strictly in HH:MM format, and keeps the stored file synchronized with every Create, Read, Update, and Delete operation performed by the user.

1.4 Objective

1.4.1 General Objectives

To design and implement a console-based weekly schedule planner in C++ that performs Create, Read, Update, and Delete (CRUD) operations using file handling, applying Object-Oriented Programming principles.

1.4.2 Specific Objectives

- To apply OOP concepts such as classes, inheritance, and encapsulation in the design of the planner.
- To implement Create, Read, Update, and Delete operations on weekly schedule data stored in a file.
- To validate user input, particularly time values, and reject improperly formatted entries.
- To provide supporting features such as keyword search, a smart reminder, and weekly time statistics.
- To persist all schedule and personal data using file handling so that records are written to disk after every change.

1.5 Significance of the Study

This project is significant as an academic exercise because it combines OOP design with file handling in a single, coherent console application. It reinforces theoretical concepts such as inheritance and encapsulation through practical implementation and results in a lightweight, genuinely usable tool for personal time management, while also providing a foundation that could be extended into a more advanced task or calendar management system.

CHAPTER 2: LITERATURE REVIEW

File handling is a foundational topic in C++ programming, typically introduced through the `fstream` library, which provides `ifstream` and `ofstream` classes for reading from and writing to files. Console-based record-management tools, whether for contacts, inventory, or schedules, commonly follow the same basic pattern: an in-memory data structure such as a vector or array holds the current set of records during program execution, and the file on disk is treated as the persistent copy of that data, rewritten whenever the in-memory structure changes.

Object-Oriented Programming provides two concepts that are especially relevant to organizing such a tool: encapsulation, which groups related data and behaviour into a class and restricts direct access to internal fields; and inheritance, which allows a more specific class to reuse and extend the fields and behaviour of a more general class. In this project, a base class holding personal details is extended by a derived class that adds weekly schedule management, so that the more specific class automatically has access to the general one without duplicating its fields. This mirrors common textbook patterns for structuring small OOP applications where a general 'profile' or 'user' class is extended by a class representing a more specific feature set.

Menu-driven console interfaces, where a loop repeatedly displays numbered options and dispatches to the corresponding function, remain a standard and simple way to expose CRUD functionality to a user without requiring a graphical interface, and are widely used in introductory and intermediate programming coursework for exactly this reason.

CHAPTER 3: METHODOLOGY

This chapter describes how the project was implemented, including the class design, project structure, the file-handling approach used for CRUD operations, input validation, and the overall program algorithm.

3.1 System Design

The system is built around two classes and one structure, in line with the OOP principles of encapsulation and inheritance.

3.1.1 Class Design

Class / Struct	Responsibility	Key Functions
Me (base class)	Collects and stores the user's personal details (name, age, class, address, gender)	displayMe()
Slot (struct)	Represents a single schedule entry: start time, end time, and activity name	-
Plan (derived from Me)	Manages the weekly schedule and short note, and performs all CRUD operations backed by file handling	addSlot(), viewSchedule(), editSlot(), deleteSlot(), searchActivities(), checkReminders(), weeklyStatistics(), manageNote(), saveToFile()

Table 3.1: Class Responsibilities

The Plan class inherits publicly from Me (class Plan : public Me), so every Plan object automatically carries the personal details collected by the Me constructor without needing to redeclare or re-collect them. Encapsulation is applied by keeping the personal detail fields and the day-wise schedule array protected, and exposing controlled access only through member functions such as displayMe(), addSlot(), and viewSchedule().

3.2 Project Structure

The program is implemented as a single source file containing the Me class, the Slot structure, the Plan class, a set of free helper functions (parseTime, promptTime, formatTime, toLower, formatHours), and a main() function that constructs a Plan object and calls its run() method to start the menu loop. All schedule and note data produced by the program is written to a single output file, plan.txt, in the working directory.

3.3 File Handling and CRUD Implementation

File handling is implemented using ofstream in the private saveToFile() member function of the Plan class, which opens plan.txt and rewrites the user's personal details, the full weekly schedule (day by day), and the short note in a formatted, human-readable layout. saveToFile() is called immediately after every operation that changes the in-memory data, so the file on disk is always kept in sync with the program's current state. The four CRUD operations map onto the program as follows:

- Create - addSlot() prompts for a day, an activity name, and a validated start and end time, appends a new Slot to the vector for that day, and calls saveToFile().
- Read - viewSchedule() and printDay() display either a single day or the full week by iterating over the in-memory vector<Slot> schedule[7] array; searchActivities() and checkReminders() are read-only operations built on top of the same data.
- Update - editSlot() locates a slot by day and index, allows the user to leave any field blank to keep its current value, re-validates any new time entered, and calls saveToFile().
- Delete - deleteSlot() locates a slot by day and index, removes it from the vector using vector::erase(), and calls saveToFile().

3.4 Input Validation

Time values are validated by the parseTime() function, which checks that the input is exactly five characters in the form HH:MM, that the third character is a colon, that the remaining characters are digits, and that the resulting hour and minute fall within valid ranges; promptTime() repeatedly asks the user for input until parseTime() returns a valid result. In editSlot(), an empty line is treated as 'keep the current value', while a non-empty but invalid time is rejected with a message and the previous value is retained.

3.5 Algorithm

- 1. Construct a Me object, collecting the user's personal details through the constructor.
- 2. Construct a Plan object, which inherits the collected personal details and initializes an empty weekly schedule.
- 3. Display a numbered menu in a loop and read the user's choice.
- 4. Create: addSlot() - choose a day, enter an activity and validated start/end times, append to the day's vector, and save to file.
- 5. Read: viewSchedule() - print either a single day or the entire week from the in-memory schedule.

- 6. Update: editSlot() - choose a day and slot number, apply any changed fields with validation, and save to file.
- 7. Delete: deleteSlot() - choose a day and slot number, remove the slot, and save to file.
- 8. Additional read-based features: searchActivities() searches all days for a keyword; checkReminders() compares a supplied time against a day's slots to report the current and next activity; weeklyStatistics() aggregates total minutes per activity using a map and prints a text-based bar chart; manageNote()/viewNote() let the user write, edit, clear, and view a short free-text note, also saved to file.
- 9. On choosing Exit, the program performs a final save to file and terminates.

3.6 Source Code Overview

The complete, compilable source code for the Me class, the Slot structure, the Plan class, all helper functions, and main() is included in Appendix A / the attached project file. The program was developed and compiled using a standard C++ compiler (g++), and uses the following standard library headers: iostream, string, fstream, vector, map, and ctype.

CHAPTER 4: RESULTS AND ANALYSIS

This chapter presents sample terminal output produced by running the program, and the results of functional testing. As no image-based screenshots were captured, the terminal output below is reproduced as text exactly as it appears when the program runs; the user may replace these blocks with actual screenshots if desired.

4.1 Results

4.1.1 Personal Details Entry Output

```
===== PERSONAL DETAILS =====  
Enter your name: Biraj Lamsal  
Enter your age: 20  
Enter your class: BCT  
Enter your address: Kathmandu  
Enter your gender: Male  
  
Hello Biraj Lamsal!  
Let's manage your weekly schedule.
```

Fig 4.1: Personal Details Entry Output

4.1.2 Adding and Viewing a Slot (Create + Read)

```
===== DAYS =====  
1. Sunday   2. Monday   3. Tuesday   4. Wednesday  
5. Thursday 6. Friday    7. Saturday 0. Cancel  
Choose a day: 2  
  
Adding to Monday  
Enter Activity: OOP Class  
Start Time (HH:MM): 09:00  
End Time (HH:MM): 10:30  
Slot added and saved.  
  
Monday  
-----  
1. 09:00 - 10:30 : OOP Class
```

Fig 4.2: Adding and Viewing a Slot (Create + Read)

4.1.3 Weekly Statistics Output

```
===== WEEKLY STATISTICS =====
```

```

00P Class
##### 1.5 hrs

Gym
##### 1 hrs

```

Fig 4.3: Weekly Statistics Output

4.1.4 Saved plan.txt File Content

```

===== PERSONAL DETAILS =====
Name   : Biraj Lamsal
Age    : 20
Class  : BCT
Address : Kathmandu
Gender : Male

===== WEEKLY SCHEDULE =====

Monday
-----
09:00 - 10:30 : 00P Class

```

Fig 4.4: Saved plan.txt File Content

4.2 Testing

The program was tested across all menu options before being finalized. All test cases passed successfully, as summarized below.

Test Case	Result
Program compilation	Passed
Personal details entry	Passed
Create - add slot with valid time	Passed
Create - reject invalid time format	Passed
Read - view single day / full week	Passed
Update - edit slot, blank keeps current value	Passed
Delete - remove slot	Passed
Search activities by keyword	Passed
Smart reminder (current / next activity)	Passed

Test Case	Result
Weekly statistics bar chart	Passed
Short note write / clear / view	Passed
File handling (plan.txt saved after every change)	Passed

Table 4.1: Testing Summary

CHAPTER 5: CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENT

5.1 Conclusion

This project successfully demonstrates the design and implementation of a console-based weekly schedule planner built in C++ using Object-Oriented Programming principles. Through the use of inheritance, encapsulation, input validation, and file handling, the application achieves reliable Create, Read, Update, and Delete operations on a weekly schedule, along with supporting features such as search, reminders, weekly statistics, and note-taking. The two-class design (Me and Plan) reinforced core OOP concepts, and the completed project was tested across all menu options to confirm correct behaviour.

5.2 Limitations

- The program does not load previously saved data from plan.txt when it starts, so each run begins with an empty schedule even if a file from a previous session already exists.
- All data is stored in a single plain-text file, which is rewritten in full on every change rather than using a structured or indexed storage format.
- Menu and day selections are read using `stoi()` without full validation, so a non-numeric entry at these prompts can cause the program to terminate unexpectedly.
- The interface is a command-line, menu-driven interface rather than a graphical user interface.
- The system supports only a single user profile per run, with no multi-user or authentication support.

5.3 Future Enhancement

Some possible future enhancements of the project are listed below:

- Load existing data from plan.txt at startup so the schedule persists correctly across program runs.
- Add more robust input validation around menu and day-selection prompts to prevent crashes on non-numeric input.
- Replace the flat text file with a structured format (e.g. CSV or a small binary/database format) for more reliable storage and easier editing.
- Develop a graphical user interface (GUI) to replace the current command-line interface.
- Add multi-user support, allowing multiple named profiles to be stored and selected.

REFERENCES

- [1] E. Balagurusamy, "Object-Oriented Programming with C++," 8th ed., McGraw-Hill Education, 2020.
- [2] B. Stroustrup, "The C++ Programming Language," 4th ed., Addison-Wesley, 2013.
- [3] H. M. Deitel and P. J. Deitel, "C++ How to Program," 10th ed., Pearson, 2017.
- [4] N. M. Josuttis, "The C++ Standard Library: A Tutorial and Reference," 2nd ed., Addison-Wesley, 2012.